

ELY NOTES

C Programming Study Pack

Programming 1 + Programming 2

A standalone, beginner-friendly version of the ELY_NOTES repository.
No Visual Studio required to read or use this guide.

Originally written by Ely • Compiled into PDF edition • 15 August 2026

00 How to Use This Pack

Read the notes anywhere; compile the examples only when you want to practice.

The GitHub repository stores the lessons as C source files. That is great for coding practice, but it can be awkward for someone who only wants to study the explanations. This PDF turns the same ideas into a normal handout while keeping runnable examples beside each topic.

Important: Visual Studio is not required. C source code only needs a C compiler to run; the editor is your choice.

Three easy ways to use the examples

- Read only: open this PDF on a phone, tablet, or computer. Nothing needs to be installed.
- Online compiler: paste a code example into any reputable online C compiler and press Run.
- Local compiler: use GCC or Clang from a terminal, with VS Code, Notepad++, Vim, or any editor you prefer.

Basic compile commands

```
# GCC (Windows / Linux)
gcc example.c -o example

# Run on Windows PowerShell
.\example.exe

# Run on Linux / macOS
./example

# Clang (commonly available on macOS)
clang example.c -o example
./example
```

Tip: treat the code as something to experiment with. Change values, break things, read the compiler error, then fix it. That is part of learning C.

00 Contents

Programming 1

- Variables
- Data Types
- Libraries / Header Files
- Input & Output
- Conditionals: if / else
- Switch Statements
- Loops
- Functions
- Arrays
- Strings
- Binary Search

Programming 2

- Pointers
- Dynamic Memory Allocation
- Structs
- Sorting
- Linked Lists

Reference

- Quick C Cheat Sheet

PROGRAMMING 1

Welcome to Programming notes - personally made by Ely. Whether you are writing your first line of code or trying to figure out why C is throwing a hundred error messages at you, the goal here is to make the core concepts clear, simple, and less stressful.

Instead of relying on dry definitions, the notes focus on bite-sized code examples and the WHY behind how things work.

- Libraries & setup
- Variables & data types
- Input & output
- Control flow
- Loops
- Functions
- Arrays & strings
- Binary search

Ely note: Take it one card at a time, run the examples, and make programming click.

01 Variables

A labeled box in memory that stores a value.

A variable gives a value a name so you can store it, read it, and change it while the program runs.

Syntax

```
dataType variableName = initialValue;
```

Naming rules

- Start with a letter or underscore; do not start with a number.
- Names are case-sensitive: myVar and myvar are different.
- Do not use reserved C keywords such as int, return, if, or while.
- Declaration creates the variable; initialization gives it a starting value.

```
#include <stdio.h>

int main(void) {
    int score = 0;
    printf("Starting score: %d\n", score);

    score = 50;
    printf("New score: %d\n", score);

    score += 10;
    printf("Final score: %d\n", score);

    return 0;
}
```

02 Data Types

Tell C what kind of value a variable stores and how it should be interpreted.

Type	Used for	Example	Common format
int	Whole numbers	19	%d
float	Decimal numbers	3.75f	%f
double	Higher-precision decimals	3.1415926535	%f for printf; %lf for scanf
char	One character	'A'	%c

Important rules

- Match the format specifier to the data type when using printf or scanf.
- Storing a decimal value in an int discards the fractional part.
- Character literals use single quotes; strings use double quotes.

```
#include <stdio.h>

int main(void) {
    int age = 19;
    float gpa = 3.75f;
    double exactPi = 3.1415926535;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("GPA: %.2f\n", gpa);
    printf("Exact Pi: %.10f\n", exactPi);
    printf("Grade: %c\n", grade);
    return 0;
}
```

03 Libraries / Header Files

Pre-written declarations and tools you can include in your program.

In C, header files are included near the top of a source file using `#include`. They give your program access to standard functions and types.

Header	Typical use
<code><stdio.h></code>	<code>printf</code> , <code>scanf</code> and standard input/output
<code><stdlib.h></code>	<code>malloc</code> , <code>realloc</code> , <code>free</code> , conversions and utilities
<code><stdbool.h></code>	<code>bool</code> , <code>true</code> and <code>false</code>
<code><string.h></code>	<code>strlen</code> , <code>strcpy</code> , <code>strcmp</code> and other string helpers
<code><math.h></code>	<code>sqrt</code> , <code>pow</code> , <code>fabs</code> and other math functions

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    // Your program logic goes here.
    return 0;
}
```

Key idea: Including a header does not mean C magically imports all code into your file; it gives the compiler the declarations it needs to use those library facilities correctly.

04 Input & Output

Using stdio.h to communicate with the user.

printf displays formatted output. scanf reads formatted input and stores it in variables.

```
#include <stdio.h>

int main(void) {
    int age;
    float height;

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height in meters: ");
    scanf("%f", &height);

    printf("You are %d years old and %.2f m tall.\n", age, height);
    return 0;
}
```

- scanf needs a format specifier such as %d, %f, %lf, or %c.
- For ordinary scalar variables, scanf usually receives the variable address using &.
- The & operator means "address of". Pointers in Programming 2 explain why this works.

05 Conditionals: if / else

Let the program make decisions based on true or false expressions.

Comparison operators

```
==  equal to
!=  not equal to
>   greater than
<   less than
>=  greater than or equal to
<=  less than or equal to

&&  AND
||   OR
!    NOT

#include <stdio.h>

int main(void) {
    int age = 18;

    if (age >= 18) {
        printf("You can vote!\n");
    } else {
        printf("Too young to vote!\n");
    }

    char choice = 'y';
    if (choice == 'Y' || choice == 'y') {
        printf("You selected YES!\n");
    }

    return 0;
}
```

Common mistake: Use == when comparing values. A single = assigns a value.

06 Switch Statements

A clean way to select between several specific values.

- switch is useful for discrete choices such as integer or character menu options.
- case labels describe the values you want to match.
- break stops execution from falling through into the next case.
- default runs when no case matches.

```
#include <stdio.h>

int main(void) {
    char grade = 'A';

    switch (grade) {
        case 'A':
            printf("Outstanding work!\n");
            break;
        case 'B':
            printf("Good job!\n");
            break;
        case 'C':
            printf("Passed, but room to improve!\n");
            break;
        case 'F':
            printf("Needs retake.\n");
            break;
        default:
            printf("Invalid grade entered!\n");
    }
    return 0;
}
```

07 Loops

Repeat code automatically instead of copying the same statements over and over.

The three common loop forms

- **for:** best when you know the repetition pattern or count.
- **while:** repeats while a condition remains true.
- **do-while:** runs the body once before checking the condition.

```
#include <stdio.h>

int main(void) {
    printf("--- For Loop ---\n");
    for (int i = 1; i <= 5; i++) {
        printf("Loop count: %d\n", i);
    }

    printf("\n--- While Loop ---\n");
    int energy = 3;
    while (energy > 0) {
        printf("Energy left: %d\n", energy);
        energy--;
    }

    printf("\n--- Do-While Loop ---\n");
    int choice = 0;
    do {
        printf("Enter 1 to exit: ");
        scanf("%d", &choice);
    } while (choice != 1);

    return 0;
}
```

Infinite loop warning: A while-loop condition must eventually become false unless you intentionally want an endless loop. Ctrl+C usually stops a terminal program.

08 Functions

Named, reusable blocks of code that do one job.

Functions reduce repetition and let you organize a program into smaller pieces.

```
#include <stdio.h>

float Ely(int num1, float num2) {
    float result = num1 + num2;
    return result;
}

int main(void) {
    float result = Ely(1, 2.5f);
    printf("Result: %.1f\n", result);
    return 0;
}
```

- Function name: Ely
- Return type: float - the kind of value sent back.
- Parameters: num1 and num2 - inputs the function receives.
- Arguments: 1 and 2.5f - actual values passed during the call.
- void is used when a function does not return a value.

09 Arrays

A fixed-size collection of values of the same type.

An array stores multiple same-type values under one name. Each item is accessed with an index, and indexing starts at 0.

Bounds rule: For an array of SIZE elements, valid indexes are 0 through SIZE - 1. Going out of bounds causes undefined behavior and can crash a program.

```
#include <stdio.h>

#define SIZE 5

int main(void) {
    int Ely[SIZE] = {10, 20, 30, 40, 50};

    Ely[0] = 99;

    for (int i = 0; i < SIZE; i++) {
        printf("Ely[%d] = %d\n", i, Ely[i]);
    }

    return 0;
}
```

10 Strings

In C, a string is a sequence of char values ending with a null terminator.

Every valid C string ends with the special character `'\0'`. If `char String[50]` stores a normal C string, it can hold at most 49 visible characters plus that terminator.

```
#include <stdio.h>
#include <stdbool.h>

bool Decider(char input) {
    return input == 'Y' || input == 'y';
}

int main(void) {
    char input;
    char text[50];

    printf("Continue? [Y/N]: ");
    scanf(" %c", &input);

    if (Decider(input)) {
        printf("Continuing!\n");
    }

    // text can store a C string of up to 49 visible characters.
    (void)text;
    return 0;
}
```

- A char stores one character; a char array can store a string.
- ASCII assigns integer codes to characters, which is why character comparisons and conversions are possible.
- Remember the `'\0'` terminator when working with C strings manually.

11 Binary Search

A fast search technique that repeatedly cuts a sorted search range in half.

Key rule: Binary search only works correctly when the data is sorted according to the same ordering used by the search.

- Find the middle element.
- If it equals the target, return the index.
- If the target is larger, discard the left half.
- If the target is smaller, discard the right half.
- Stop when low becomes greater than high.

```
#include <stdio.h>

int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    int numbers[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int target = 23;

    int result = binarySearch(numbers, size, target);
    if (result != -1)
        printf("Found %d at index %d!\n", target, result);
    else
        printf("%d was not found.\n", target);

    return 0;
}
```

PROGRAMMING 2

Congratulations on getting through the Programming 1 foundations. Programming 2 is where C starts exposing how memory and data structures really work.

- Pointers & memory handling
- Dynamic allocation with malloc, calloc, realloc, and free
- Structs & custom data types
- Sorting algorithms
- Linked lists

Ely note: Code along, do not be afraid to break things, and pay attention to what is happening in memory.

12 Pointers

A pointer stores a memory address.

If variable A stores 100, a pointer to A stores the address where A lives. Dereferencing the pointer with * accesses the value at that address.

```
#include <stdio.h>

void tryToChange(int num) {
    num = 100;          // changes only the local copy
}

void actuallyChange(int *numPtr) {
    *numPtr = 100;      // changes the original value through its address
}

int main(void) {
    int num = 1;

    tryToChange(num);
    printf("After tryToChange: %d\n", num);

    actuallyChange(&num);
    printf("After actuallyChange: %d\n", num);

    return 0;
}
```

- &num means "the address of num".
- int *p declares p as a pointer to int.
- *p dereferences p and accesses the int stored at the pointed-to address.
- Pointers let functions modify caller-owned data and are essential for dynamic memory and linked structures.

13 Dynamic Memory Allocation

Request memory at runtime when a fixed-size array is not enough.

Automatic local variables are managed for you, while dynamically allocated memory must be managed manually. `malloc` allocates bytes, `realloc` resizes an existing allocation, and `free` releases it.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int size = 5;
    int *ely = malloc((size_t)size * sizeof *ely);

    if (ely == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < size; i++) {
        ely[i] = (i + 1) * 10;
    }

    int newSize = 3;
    int *temp = realloc(ely, (size_t)newSize * sizeof *ely);
    if (temp != NULL) {
        ely = temp;
        size = newSize;
    }

    for (int i = 0; i < size; i++) {
        printf("%d ", ely[i]);
    }
    printf("\n");

    free(ely);
    ely = NULL;
    return 0;
}
```

Safety: Always check allocation results. Using a temporary pointer for `realloc` prevents you from losing the original allocation if resizing fails.

- `malloc` returns a pointer to the allocated block or `NULL` on failure.
- `sizeof *pointer` is a useful pattern because it automatically matches the pointed-to type.
- free memory exactly when you are done with it; do not use it after `free`.

14 Structs

Group related values of different types into one custom record.

A struct can combine fields such as name, age, and weight. typedef can give the struct type a shorter alias.

```
#include <stdio.h>
#include <string.h>

typedef struct Person {
    int age;
    float weight;
    char name[50];
} Person;

int main(void) {
    Person person1;

    person1.age = 18;
    person1.weight = 65.5f;
    strcpy(person1.name, "Ely");

    printf("Name: %s\n", person1.name);
    printf("Age: %d\n", person1.age);
    printf("Weight: %.2f kg\n", person1.weight);
    return 0;
}
```

- Use the dot operator (.) with an ordinary struct variable.
- When you have a pointer to a struct, the arrow operator (->) is the usual shorthand.
- Structs are the building blocks for more complex data structures such as linked-list nodes.

15 Sorting

Rearrange data into a defined order so it is easier to process and search.

- Bubble sort: repeatedly swaps adjacent out-of-order values. Easy to learn, $O(n^2)$ in the typical case.
- Selection sort: repeatedly selects the smallest remaining value.
- Insertion sort: grows a sorted prefix one item at a time.
- More advanced algorithms include merge sort and quicksort.

```
#include <stdio.h>

void bubbleSort(int arr[], int size) {
    int swapped;

    do {
        swapped = 0;
        for (int i = 0; i < size - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                swapped = 1;
            }
        }
    } while (swapped);
}

int main(void) {
    int numbers[] = {5, 8, 3, 1, 2};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    bubbleSort(numbers, size);

    for (int i = 0; i < size; i++)
        printf("%d ", numbers[i]);
    printf("\n");
    return 0;
}
```

Study trick: Sorting becomes much easier when you draw the array after every comparison or watch an algorithm visualizer.

16 Linked Lists

Nodes stored separately in memory and connected using pointers.

Unlike an array, linked-list nodes do not need to sit next to each other in one contiguous block. Each node stores data and a pointer to the next node.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = malloc(sizeof *head);
    Node *second = malloc(sizeof *second);

    if (head == NULL || second == NULL) {
        free(head);
        free(second);
        return 1;
    }

    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = NULL;

    for (Node *current = head; current != NULL; current = current->next) {
        printf("[%d] -> ", current->data);
    }
    printf("NULL\n");

    free(second);
    free(head);
    return 0;
}
```

[Head] -> [10 | next] -> [20 | next] -> NULL

- head points to the first node.
- next points to the following node.
- NULL marks the end of this simple singly linked list.
- Traversal follows next pointers until NULL.
- Every dynamically allocated node must eventually be freed.

17 Quick C Cheat Sheet

The symbols and patterns you will keep seeing.

Concept	Pattern	Meaning
Variable	<code>int x = 10;</code>	Stores a named value.
Address-of	<code>&x</code>	Gets the memory address of x.
Pointer	<code>int *p = &x;</code>	p stores the address of an int.
Dereference	<code>*p</code>	Accesses the value at the address stored in p.
Array access	<code>arr[i]</code>	Gets element at index i.
Struct field	<code>person.age</code>	Access field through a struct variable.
Struct pointer field	<code>node->next</code>	Access field through a pointer to struct.
Allocate	<code>malloc(n * sizeof *p)</code>	Requests dynamic memory.
Resize	<code>realloc(p, newBytes)</code>	Attempts to resize an allocation.
Release	<code>free(p)</code>	Returns dynamic memory.
Conditional	<code>if (condition) { ... }</code>	Runs code when expression is true.
Loop	<code>while (condition) { ... }</code>	Repeats while condition is true.

That's the current ELY NOTES pack.

Read it. Run it. Break it. Fix it. Repeat.

Source repository: github.com/MarkStock-it/ELY_NOTES